

PrivateFin: A Privacy-First, Explainable Financial Advisory System

Template: CM3020 Artificial Intelligence — Project Idea 2: Financial Advisor Bot

Public repository: <https://github.com/dsarney/privatefin>

Table of Contents

Chapter 1: Introduction.....	5
1.1 Project context and problem.....	5
1.2 Motivation and research gap.....	5
1.3 Aim, objectives and research questions.....	5
1.4 Scope, limitations and report structure.....	6
Chapter 2: Literature Review.....	7
2.1 Review framing.....	7
2.2 Domain-specific language models.....	7
2.3 Quantitative grounding and data provenance.....	8
2.4 Explainability and generative risk.....	9
2.5 Regulatory and ethical context.....	9
2.6 Alternative methodologies.....	10
2.7 Synthesis and research gap.....	11
Chapter 3: Design.....	12
3.1 Design objectives and requirements.....	12
3.2 Users and interaction design.....	12
3.3 System architecture and data flow.....	14
3.4 Component design.....	14
3.4.1 Data ingestion.....	14
3.4.2 Indicator engine.....	15
3.4.3 Prompt and inference.....	15
3.5 Implemented design versus planned work.....	16
3.6 Evaluation-oriented design and contingencies.....	16
Chapter 4: Implementation.....	18
4.1 Implementation overview.....	18
4.2 Data acquisition and persistence.....	18
4.3 Indicator engine.....	19
4.4 Structured prompt construction.....	20
4.5 Local model client and failure containment.....	20
4.6 Streamlit interface and end-to-end flow.....	21

4.7 Verification implemented in the repository.....	21
Chapter 5: Evaluation.....	23
5.1 Aims, evidence and decision rules.....	23
5.2 Aim 1: implementation reliability.....	23
Method.....	23
Results.....	23
Interpretation.....	23
Threats.....	23
5.3 Aim 2: independent indicator agreement.....	24
Method.....	24
Results.....	24
Interpretation.....	24
Threats.....	25
5.4 Aim 3: local-inference privacy.....	25
Method.....	25
Results.....	25
Interpretation.....	26
Threats.....	26
5.5 Aim 4: OIIR explanation quality and faithfulness.....	26
Method.....	26
Results.....	26
Interpretation.....	27
Threats.....	27
5.6 Aim 5: usability with target users.....	27
Method.....	27
Results.....	27
Interpretation.....	27
Threats.....	28
5.7 Integrated interpretation and limitations.....	28
Chapter 6: Conclusion.....	29
6.1 Project outcome.....	29
6.2 Contribution, limitations and future work.....	29
6.3 Final judgement.....	30

References.....	31
-----------------	----

Chapter 1: Introduction

1.1 Project context and problem

Artificial intelligence is increasingly used to interpret financial information, yet contemporary systems create barriers for retail users. A cloud-hosted model can convert market data into fluent advice, but its query may reveal securities under consideration, risk tolerance and financial intentions. The recommendation can also appear authoritative without exposing a verifiable relationship between evidence and conclusion. PrivateFin investigates whether a useful alternative can combine deterministic technical analysis with locally executed language-model inference.

The project is developed under the exact template statement “CM3020 Artificial Intelligence — Project Idea 2: Financial Advisor Bot”. PrivateFin treats it as an explanation problem rather than automated trading. It does not predict prices with certainty or execute trades; it retrieves historical equity data, calculates transparent indicators and explains them cautiously. The prototype uses `yfinance`, `Pandas`, a quantised Llama 3 model served through Ollama, and Streamlit.

Privacy is claimed narrowly. The ticker, indicators, date range and risk profile are sent to an Ollama loopback service, not intentionally to a cloud LLM provider. The system is not wholly offline: OHLCV retrieval creates Yahoo Finance egress, exposing request metadata. No API key reduces account linkage but does not ensure anonymity. A ten-query endpoint audit observed only loopback sockets for the Python and Ollama processes, but it did not inspect packet payloads. This report therefore uses “local LLM inference”, not “zero system egress”.

1.2 Motivation and research gap

Existing financial language technologies expose a trade-off. BloombergGPT demonstrates the performance benefits of finance-weighted training but is embedded in a proprietary ecosystem (Wu et al., 2023). FinGPT promotes open financial language-model research and parameter-efficient adaptation, yet common workflows still depend on externally hosted data or inference services (Yang et al., 2023). FinBERT shows that financial-domain pre-training improves sentiment classification, but classification labels cannot by themselves provide a generative, user-specific explanation (Araci, 2019). At the other extreme, deterministic indicators are inspectable but difficult for novices to interpret and cannot express uncertainty or contextual caution.

PrivateFin addresses this gap through a hybrid symbolic–generative design. Closed-form calculations remain the quantitative source of truth, while the language model is constrained to communicate supplied values through an Observe–Interpret–Infer–Recommend (OIIR) structure. This arrangement does not make generated prose inherently faithful: large language models can still misstate an indicator despite receiving the correct number (Bender et al., 2021). It does, however, create a traceable comparison point from which such errors can be detected. The intended contribution is therefore architectural synthesis—local inference, deterministic grounding and structured explanation for consumer hardware—rather than a new forecasting algorithm.

1.3 Aim, objectives and research questions

The project aim is to design and evaluate a locally operated financial advisory prototype that turns reproducible market indicators into understandable, evidence-linked guidance while limiting disclosure of user query data during language-model inference.

This aim is operationalised through five objectives:

1. retrieve historical equity data for a user-selected company and period;
2. calculate RSI(14) and MACD(12/26/9) reproducibly and expose their values to the user;
3. generate a four-part OIIR explanation grounded in those calculated values;
4. keep LLM prompts and responses on the host device through Ollama, while documenting unavoidable Yahoo Finance egress;
5. provide a browser interface, educational disclaimer and fallback output suitable for non-technical users.

Three research questions organise the later evaluation. **RQ1:** To what extent does the implementation preserve query privacy during local LLM inference, given that market-data acquisition remains external? **RQ2:** Do the implemented indicator calculations behave consistently and agree with an independent reference under a declared tolerance? **RQ3:** Are OIIR explanations complete, logically proportionate and faithful to the supplied data for novice users? These questions deliberately separate architecture, numerical correctness and communicative quality; evidence for one cannot substitute for another.

A successful outcome therefore requires more than a working demonstration. It requires evidence that each boundary behaves as designed, that prose remains subordinate to quantitative evidence, and that users can distinguish an educational interpretation from a personalised instruction to trade.

1.4 Scope, limitations and report structure

PrivateFin analyses one selected company at a time from a fixed list of 108 options. For Alphabet, it produces a synthetic average of GOOG and GOOGL OHLCV series rather than analysing a directly tradable instrument. It uses only historical prices, RSI and MACD; it does not incorporate fundamentals, news, fees, taxation, portfolio diversification or individual financial circumstances. Outputs are educational and are not professional financial advice. The project does not claim that indicator-based narratives predict returns, satisfy suitability obligations, or make the application a regulated adviser.

The evidence is bounded. Eighteen module tests have passed, but UI automation is absent. TA-Lib comparisons require attention to differing initialisation conventions. A completed 20-output OIIR assessment found perfect structure compliance but a single-author mean of 5.95/8, narrowly below the pre-declared 6/8 target. A ten-query endpoint audit observed only loopback sockets for the Python and Ollama processes, although it did not inspect packet payloads. No participant feedback, SUS result or clarity score exists.

Chapter 2 critically evaluates the technical, regulatory and methodological literature from which the research gap is derived. Chapter 3 translates the aim into requirements and presents the implemented architecture, interface and component boundaries while distinguishing current functionality from planned evaluation and enhancements. Chapter 4 explains the implemented pipeline, Chapter 5 evaluates the available evidence and Chapter 6 gives the final judgement and priorities for further work.

Chapter 2: Literature Review

2.1 Review framing

PrivateFin sits at the intersection of financial natural-language processing, quantitative technical analysis, privacy engineering and explainable AI. These areas cannot be assessed independently. A model may score highly on a financial benchmark yet remain unsuitable for retail advice because its deployment discloses queries or its conclusions cannot be checked. Conversely, a transparent indicator may be mathematically reproducible but practically inaccessible to a novice. The relevant question is therefore not which prior system is “best”, but which combination of methods supports PrivateFin’s narrower goals: evidence-linked explanation, local LLM inference, feasible execution on consumer hardware and honest treatment of uncertainty.

2.2 Domain-specific language models

BloombergGPT is a 50-billion-parameter decoder-only model trained on FinPile, a 363-billion-token corpus combining proprietary financial material with general text (Wu et al., 2023). Its most important result for this project is methodological rather than commercial: weighting training data toward a specialist domain can outperform similarly sized general models on tasks such as financial sentiment, question answering and named-entity recognition. This weakens the assumption that scale alone determines financial capability. Nevertheless, BloombergGPT is not an appropriate implementation model for PrivateFin. Its data and computational requirements are incompatible with a student project and consumer hardware, while its proprietary context limits reproducibility and retail access. The paper also evaluates task performance rather than a user-facing mechanism for tracing advisory prose back to independently calculated indicators. It demonstrates financial language competence, not the complete privacy-and-explanation proposition examined here.

FinGPT offers a contrasting open-source strategy. It combines financial data pipelines with foundation models and uses Low-Rank Adaptation (LoRA) to reduce the cost of domain fine-tuning (Yang et al., 2023). LoRA represents weight updates through low-rank matrices, drastically reducing the number of trainable parameters while retaining a frozen base model (Hu et al., 2022). This is important because it makes adaptation more accessible than Bloomberg-scale pre-training. FinGPT also recognises that financial knowledge changes quickly and therefore emphasises live data sources.

Openness at the model or code level, however, should not be conflated with private deployment. FinGPT workflows may consume news, social media and hosted services, and whether inference is local depends on the selected implementation. Its heterogeneous inputs can improve timeliness but introduce provenance and reliability problems: a regulatory filing and an anonymous social-media post should not carry equivalent evidential weight. FinGPT consequently informs PrivateFin in two opposing ways. Its efficient specialisation supports the principle that a narrowly structured context can be valuable, but its broad data ambitions are rejected in favour of a small, auditable market-data surface. PrivateFin uses prompt constraints rather than fine-tuning, accepting lower domain specialisation in exchange for feasibility and easier inspection.

FinBERT provides stronger evidence for domain adaptation in a discriminative setting. Araci (2019) continued pre-training BERT on financial text before fine-tuning it for sentiment analysis. BERT’s bidirectional representations are well suited to classification because each token is contextualised

using surrounding language (Devlin et al., 2019). FinBERT substantially outperformed general sentiment baselines on Financial PhraseBank, whose annotations capture domain-specific polarity that ordinary lexicons can mishandle (Malo et al., 2014). This supports the broader claim that financial terminology and rhetorical conventions require specialist treatment.

Yet sentiment classification and financial advice are not interchangeable. A positive, neutral or negative label neither calculates market momentum nor explains what a user should infer from it. An encoder-only classifier can be more stable and easier to benchmark than a generative model, but it lacks the flexible natural-language output required by the project template. PrivateFin therefore adopts FinBERT’s domain-grounding lesson without adopting its task definition. It supplies explicit financial values and plain-English classifications to a decoder model, attempting to narrow the model’s interpretive freedom while retaining generative communication. This is a design compromise, not evidence that prompting reproduces the benefits of domain-adaptive pre-training.

2.3 Quantitative grounding and data provenance

The quantitative layer uses `yfinance` to request Open, High, Low, Close and Volume data from Yahoo Finance (Aroussi, 2019), represented and transformed through Pandas (McKinney, 2010). The library is attractive because it is accessible without a paid account and returns tabular data suitable for reproducible computation. For a coursework prototype, this reduces cost and enables a reader to repeat a historical calculation. However, `yfinance` is an unofficial interface with no service-level guarantee. Data can be unavailable, revised or shaped differently across library versions. Repeating identical parameters is therefore not an unconditional guarantee of identical results over time. Local CSV persistence improves auditability by preserving the data actually used, but it does not establish the upstream dataset’s correctness.

The no-key interface also has a qualified privacy benefit. It avoids placing an account identifier directly in an API request, yet the request still leaves the device and may expose IP address, ticker, timing and other metadata to Yahoo Finance. This distinction is central to the project: the LLM interaction is designed to remain local, whereas market-data acquisition is intentionally external. A privacy assessment must observe both flows rather than treating the whole application as offline.

RSI and MACD were selected because their calculations are established, compact and inspectable. Wilder’s RSI compares smoothed gains with smoothed losses, conventionally over fourteen periods, and maps relative strength to a value between zero and one hundred (Wilder, 1978). Values at or below 30 are commonly read as oversold and values at or above 70 as overbought, although these are heuristics rather than natural laws. MACD subtracts a 26-period exponential moving average from a 12-period average and compares the result with a nine-period signal line (Appel, 1979). The sign and crossover relationship provide a compact description of changing momentum.

These indicators support auditability but not predictive certainty. Threshold meanings depend on asset, horizon and market regime; an “overbought” reading can persist during a strong rise and does not mechanically imply an imminent fall. RSI and MACD are also derived from the same price history, so presenting two outputs does not create two independent sources of evidence. PrivateFin must consequently describe them as limited momentum signals and make recommendations proportionate to the data. Their principal research value is that they form a

deterministic reference against which generated statements can be checked. They should not be used to imply that a transparent heuristic is necessarily an accurate investment strategy.

2.4 Explainability and generative risk

Rudin (2019) argues that high-stakes applications should favour inherently interpretable models over post-hoc explanations of opaque predictors. PrivateFin cannot fully satisfy this preference because Llama 3 remains a generative neural model. Its response is not an interpretable model in Rudin's strict sense, and a fluent OIIR narrative is not proof of the causal process that produced each token. The architecture instead confines the black box to communication: RSI and MACD are calculated outside the model, supplied explicitly, and remain visible for comparison. This is more defensible than asking a language model both to invent the quantitative analysis and explain it, but it does not eliminate opacity.

Bender et al. (2021) emphasise that large language models can produce plausible text without grounded understanding. In financial advice, fluency can amplify harm because users may treat confident language as expertise. PrivateFin's known prototype error—describing RSI 58.75 as leaning oversold despite the supplied balanced classification—illustrates this risk. The error did not originate in indicator mathematics; it occurred while converting evidence into prose. Structured prompting can improve format compliance, but format is distinct from faithfulness. An output may contain all four OIIR headings and still misquote a value, apply the wrong threshold or recommend an action stronger than the evidence supports.

Chain-of-Thought prompting shows that intermediate reasoning prompts can improve performance on some multi-step tasks (Wei et al., 2022). PrivateFin's OIIR scaffold is related but should not be mischaracterised as access to hidden model reasoning. It requests a concise, user-facing rationale—what was observed, how it was interpreted, what was inferred and what cautious action follows—and explicitly avoids disclosure claims about internal chain-of-thought. OIIR is useful because it standardises the explanation surface and enables rubric scoring. It cannot guarantee that the visible rationale faithfully represents the model's internal computation. Automatic comparison of quoted numbers and classifications with the indicator dictionary is therefore a necessary planned safeguard.

Llama 3 is suitable for local experimentation because open-weight variants can be quantised and run through consumer-oriented tooling (Grattafiori et al., 2024). Local execution changes the privacy boundary by keeping prompts at the Ollama loopback service, but it does not improve factuality by itself. Quantisation may also alter output quality, and unspecified sampling settings can make repeated responses differ. The local model should therefore be understood as a controllable deployment choice, not a validated financial expert.

2.5 Regulatory and ethical context

The EU Artificial Intelligence Act provides a useful transparency benchmark but must be applied accurately. Retail investment advice is not itself listed as a high-risk use case in Annex III; finance-adjacent categories include creditworthiness and risk assessment or pricing in life and health insurance (European Union, 2024). PrivateFin does not claim high-risk status or legal compliance certification. It voluntarily draws on Article 13's emphasis on sufficient transparency for deployers to interpret outputs and on Article 14's human-oversight principle. OIIR, visible metrics, cautious language and an educational disclaimer are design responses to those principles. Whether they

achieve “meaningful transparency” is an empirical question for evaluation, not a conclusion established by adopting headings.

GDPR similarly shapes the architecture without automatically determining legal status. Article 5 includes data-minimisation principles and Article 25 requires data protection by design and by default (European Union, 2016). Sending an advisory prompt to a cloud model would add a processor and transmission pathway; local Ollama inference avoids that pathway. Nevertheless, ticker requests still reach Yahoo Finance, cached files remain on the host, and a risk profile may be personal data when linked to an identifiable person. PrivateFin currently has no account system and does not ask for holdings, but privacy engineering must consider network metadata, local storage and logs rather than only model hosting.

Article 22 concerns decisions based solely on automated processing that produce legal or similarly significant effects. PrivateFin provides educational output that a human may ignore and does not execute transactions, so its prototype use is unlikely to fit the simplest reading of a solely automated decision. Financial language can still influence consequential behaviour, however. Human oversight cannot be reduced to placing a person after the algorithm; the person needs understandable evidence, uncertainty and freedom to reject the recommendation. The disclaimer is therefore necessary but insufficient. The design must avoid false precision and distinguish observation from recommendation.

2.6 Alternative methodologies

The project template suggests techniques including reinforcement learning, evolutionary strategies and backtesting. An RL trading agent could formulate buy, hold and sell actions within a Markov decision process and optimise reward based on simulated return. Evolutionary methods could search over trading-rule parameters, while supervised neural forecasting could estimate future price or direction. These methods would create a more ambitious predictive core than two technical indicators and may be appropriate where profitability is the primary outcome.

They are poorly aligned with PrivateFin’s present research questions. A policy network’s weights do not provide a novice-readable route from observation to action, and post-hoc prose generated around a policy would risk rationalising rather than explaining it. Training and validation also create methodological hazards. Financial time series are non-stationary, and repeated strategy selection against historical data encourages overfitting, leakage and backtest selection bias (López de Prado, 2018). A high in-sample Sharpe ratio would not demonstrate that prompts stay local or explanations quote indicators faithfully. Conversely, a transparent explanation does not demonstrate profitability. The evaluation metric must match the system claim.

This does not make RL or backtesting intrinsically unsuitable. It means they answer a different question: whether a strategy learns actions associated with historical or simulated returns. If PrivateFin later issues discrete trading signals or manages portfolios, out-of-sample and transaction-cost-aware backtesting would become essential. For the current advisory prototype, deterministic indicators allow the work to concentrate on correctness, privacy boundaries and explanation quality. The cost is limited analytical depth, which must be acknowledged rather than reframed as superior forecasting.

2.7 Synthesis and research gap

The literature produces four conclusions. First, BloombergGPT, FinGPT and FinBERT show that financial specialisation matters, but benchmark competence does not establish retail accessibility, deployment privacy or explanation faithfulness (Araci, 2019; Wu et al., 2023; Yang et al., 2023). Second, RSI and MACD offer inspectable inputs, yet their conventional thresholds are context-sensitive heuristics rather than forecasts (Appel, 1979; Wilder, 1978). Third, structured generative output can make evidence easier to inspect, but a rationale remains vulnerable to hallucination and should not be equated with an inherently interpretable model (Bender et al., 2021; Rudin, 2019). Fourth, regulation-inspired transparency and privacy-by-design are useful design constraints only when claims are bounded and tested (European Union, 2016; European Union, 2024).

No reviewed comparator combines, in the specific form required here, a browser-based retail interface, locally served open-weight LLM, externally verifiable indicator calculations and a fixed OIIR explanation contract. This is a scoped research gap rather than a claim that no local financial tools exist. PrivateFin’s response is a hybrid pipeline in which external data acquisition, local deterministic computation, local generative interpretation and user presentation have explicit boundaries. Its originality lies in integrating these established elements and making tensions visible: local inference without pretending the system is offline; structured explanation without equating prose with faithfulness; and deterministic grounding without claiming profitable prediction.

The resulting methodology is proportionate to a final-year prototype. It is technically feasible on consumer hardware, supports module-level testing, and permits each central claim to be evaluated separately. Its principal weakness is that prompt engineering carries less domain knowledge than fine-tuning and less predictive ambition than learned trading strategies. That weakness defines the evaluation burden: the project must demonstrate that its narrower claims—correct calculations, bounded data flows and understandable, evidence-linked narratives—are actually met.

Chapter 3: Design

3.1 Design objectives and requirements

The design translates the research questions into separable responsibilities. The data layer must acquire and preserve the market observations used; the indicator layer must calculate values without model involvement; the prompt layer must expose those values in a stable explanation contract; the inference layer must communicate only with local Ollama; and the interface must make results and limitations understandable. Separation is important because a successful LLM response cannot validate an incorrect RSI calculation, while correct indicators cannot prove that network privacy or usability requirements have been met.

The implemented functional requirements are:

- **FR1 — Data acquisition:** accept a selected company and valid date range, retrieve OHLCV history, and handle empty responses without crashing.
- **FR2 — Indicator calculation:** compute RSI(14), MACD(12/26/9), its signal line and histogram from closing prices.
- **FR3 — Quantitative summary:** expose latest close, one-session percentage change, indicator values, and bounded trend and momentum classifications.
- **FR4 — Explanation contract:** construct a prompt containing source symbols, period, sample size, risk profile and all summary values, requesting Observe, Interpret, Infer and Recommend sections.
- **FR5 — Local generation:** send system and user prompts to Llama 3 through Ollama at `localhost:11434`.
- **FR6 — User presentation:** display controls, four metric cards, a closing-price chart, model output and an educational disclaimer in Streamlit.
- **FR7 — Graceful degradation:** retain the quantitative display and show a structured fallback template when Ollama is unavailable.

The non-functional requirements are:

- **NFR1 — Bounded privacy:** no intentional cloud transmission of the LLM prompt or response; Yahoo Finance egress must be disclosed and empirical packet capture remains required.
- **NFR2 — Reproducibility:** calculations must be deterministic for a preserved input dataset and independently comparable under documented initialisation conventions.
- **NFR3 — Accessibility:** core output must use plain English and require no programming interaction after local launch.
- **NFR4 — Traceability:** displayed and prompted values must originate from the same indicator summary, permitting later numeric faithfulness checks.
- **NFR5 — Resilience:** unavailable data or local model service must produce actionable UI feedback rather than an unhandled failure.
- **NFR6 — Ethical presentation:** every session must state that output is educational, not professional advice or endorsement.

3.2 Users and interaction design

Two personas guide the interface. The first is a self-directed retail investor who understands prices and basic market terminology but does not have a professional data terminal. This user values

rapid comparison, local control and an explanation that can be checked against visible numbers. The second is a novice who may not know what RSI or MACD means and is especially vulnerable to confident but unsupported recommendations. This user needs ordinary language, consistent structure and explicit caution. Neither persona requires automated execution, portfolio optimisation or a prediction presented as certainty.

The implemented workflow is intentionally short. A user selects one of 108 company options, chooses start and end dates, selects Low, Medium or High risk, and presses **Generate analysis**. The main area presents Latest Close, RSI(14), MACD and Price Change, followed by a price-history chart and either a generated OIIR response or a fallback scaffold. A persistent disclaimer appears before analysis. Input validation rejects a start date that is not earlier than the end date, and an empty download produces a user-facing message.

The risk profile is implemented only as a labelled prompt field. There are no explicit per-profile recommendation rules, no suitability questionnaire and no guarantee that the model meaningfully differentiates Low, Medium and High. It should therefore be described as contextual tone input, not personalisation or regulated risk assessment. Similarly, the prompt is constructed internally but is not displayed in the current Streamlit interface. A prompt-inspection pane, richer time-horizon control and model selection appeared in earlier plans but are not implemented here.



Figure 3.1 — PrivateFin sidebar and generated-analysis view. The screenshot demonstrates the implemented company, date and risk controls together with a generated OIIR response.

Latest Close	RSI(14)	MACD	Price Change
316.22	63.86	3.9634	0.90%

Figure 3.2 — Visible deterministic outputs used to ground the advisory narrative. The same summary values are displayed to the user and supplied to the prompt builder.

3.3 System architecture and data flow

PrivateFin uses a four-stage pipeline orchestrated by `src/ui/app.py`:

6. **Data ingestion:** `src/engine/data_loader.py` requests each selected ticker through `yfinance`, normalises returned columns and optionally saves the resulting frame as CSV.

7. **Indicator engine:** `src/engine/indicator_engine.py` adds RSI and MACD series and creates one flat summary dictionary.
8. **Prompt construction:** `src/llm/prompt_builder.py` combines the summary with provenance and user context under a fixed system instruction.
9. **Local inference:** `src/llm/ollama_client.py` sends explicit system and user messages to Llama 3 through the loopback Ollama service.

The Streamlit layer renders the same summary passed to the prompt builder, reducing the risk that the displayed values differ from those interpreted by the model. The model cannot modify the DataFrame or indicator functions; its authority is confined to generated text. If it is unavailable, the first three stages still produce usable metrics and the interface substitutes a generic OIIR template.

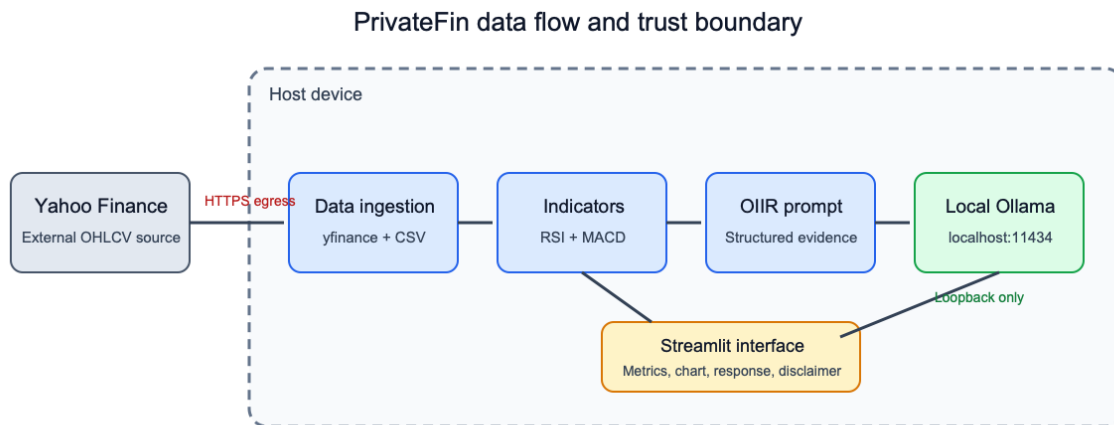


Figure 3.3 — PrivateFin trust-boundary architecture. Yahoo Finance OHLCV requests cross the host boundary; indicator computation, prompt construction and Llama 3 inference remain local by design.

The normal data flow begins with dates and symbols, continues through an OHLCV DataFrame, an enriched DataFrame and a typed summary, and ends with prompts and response text. The summary includes close, RSI, MACD, signal, histogram, percentage change, trend and momentum. This narrow contract improves testability and creates the future verification point for checking whether numbers quoted by the model match the source.

3.4 Component design

3.4.1 Data ingestion

`download_stock_data()` accepts either one ticker or a sequence. `yf.download()` is called separately for each symbol, and MultiIndex columns are flattened before downstream calculation. Empty frames are removed; if no frame remains, the function returns `None`. Successful data can be written to `data/{symbols}_historical.csv`, preserving the exact input for later audit even though repeat live requests are not yet prevented by caching.

For Alphabet, `GOOG` and `GOOGL` frames are aligned by date and arithmetic means are calculated for common OHLCV columns. This offers one company-level view but creates a synthetic series that cannot itself be traded and gives equal weight to share classes. The interface exposes the underlying symbols, and the report must retain this limitation. A future design could require the user to select a class instead.

3.4.2 Indicator engine

`compute_rsi()` derives price changes, separates gains and losses, and applies exponentially weighted means with `alpha 1/period`, `adjust=False` and a fourteen-observation warm-up. Zero-loss and zero-gain cases are clamped to 100 and 0, while remaining missing warm-up values are filled with 50. The order means a completely flat series resolves to 0 rather than the more intuitive neutral 50; this is a known implementation gap. `compute_macd()` uses 12- and 26-period exponential means, a nine-period signal and histogram equal to MACD minus signal.

`summarize_latest_indicators()` compares MACD with its signal to classify upward, downward or neutral tendency and applies RSI boundaries of 30 and 70 to classify momentum. These labels are aids to communication, not trading instructions. Keeping them in deterministic code rather than asking the model to derive them reduces, but does not remove, interpretation error.

3.4.3 Prompt and inference

The system prompt requires exactly four conceptual stages, evidence grounding, caution and explanations of technical terminology. The user prompt includes all numerical fields plus plain-English descriptions such as “recent momentum is leaning upward”. Raw and translated forms are both supplied so that outputs remain precise while accessible. The instruction explicitly avoids claims about hidden chain-of-thought; OIIR is the user-facing explanation format.

`OllamaClient` defaults to model `llama3` and host `http://localhost:11434`. Availability is checked with a lightweight local list call. Generation uses explicit system and user roles and accommodates dictionary- and object-shaped SDK responses. Exceptions become an actionable runtime error, although the UI normally detects unavailability first. Sampling temperature is not explicitly configured, so narrative reproducibility is not currently guaranteed.

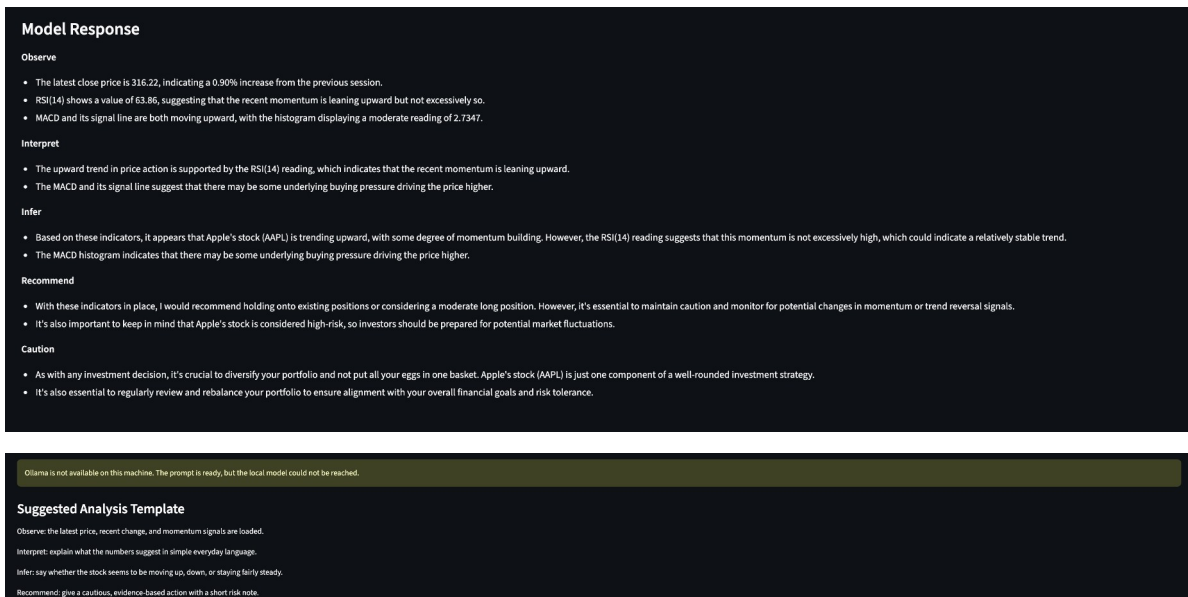


Figure 3.4 — Implemented local-model response and graceful-degradation paths. The fallback is explicitly labelled and does not pretend to be a generated interpretation of the displayed values.

3.5 Implemented design versus planned work

The working design currently includes live Yahoo Finance retrieval, CSV persistence, RSI and MACD computation, the 108-company picker, three risk labels, Streamlit metrics and charting, OIIR

prompting, local Ollama generation, disclaimers and fallback behaviour. Module tests cover data loading, indicators, prompt fields and Ollama client contracts. These features can be described as implemented, although UI orchestration remains manually tested.

Several elements remain planned and must not be presented as completed: prompt display in the interface; explicit model temperature; download caching; a minimum data-window validator; additional indicators; a SQLite audit log; comprehensive post-generation semantic validation; and Streamlit UI tests. The 20-output OIIR assessment and ten-query endpoint audit are reported in Chapter 5, but neither included an independent scorer or packet-payload inspection. The participant study remains incomplete and no participant feedback has been supplied.

The canonical source repository is <https://github.com/dsarney/privatefin>. It contains the implementation, tests and reproducible evaluation scripts referenced in this report.

3.6 Evaluation-oriented design and contingencies

Each claim has a corresponding method. Numerical correctness is assessed first through the existing eighteen unit tests and then through independent RSI/MACD comparison, with differences interpreted in light of warm-up and initialisation conventions. Privacy is assessed through repeated process-endpoint snapshots during ten frozen-prompt inferences; expected loopback Ollama traffic is separated from any external socket, while payload-level packet capture remains desirable. Explainability is assessed over 20 outputs using factual accuracy, logical coherence, OIIR completeness and transparency, supplemented by rule-based value and threshold checks. Usability requires real participants completing the prepared SUS, clarity scale and interview protocol; until then no usability outcome can be reported (Brooke, 1996). Multiple methods are necessary because software tests, expert scoring and user reception measure different properties (Manning et al., 2008).

Contingencies preserve the research aim rather than adding unsupported features. If Llama 3 produces weak OIIR structure, the prompt can be split into observation/interpretation and inference/recommendation stages, though this increases latency and the surface for inconsistency. If Yahoo Finance becomes unavailable, the ingestion interface permits replacement, but an alternative provider would require a new privacy analysis. If consumer hardware struggles with the model, a smaller or more aggressively quantised variant can be used and documented. If participant recruitment is insufficient, the study remains incomplete; expert scoring may expand but must not be described as participant evidence. This design keeps implemented functionality, planned verification and future enhancement visibly distinct.

Chapter 4: Implementation

4.1 Implementation overview

PrivateFin was implemented as a local Python application that separates quantitative processing from generative explanation. Its executable path is deliberately short: a root-level `app.py` imports and invokes the Streamlit application in `src/ui/app.py`; the interface requests market data from `src/engine/data_loader.py`, passes the returned prices to `src/engine/indicator_engine.py`, builds a structured advisory prompt through `src/llm/prompt_builder.py`, and, when available, submits that prompt to a locally addressed Ollama service through `src/llm/ollama_client.py`. This modular arrangement makes the boundary between factual calculation and language generation explicit. RSI and MACD are calculated before the language model is involved, and the model receives their values as immutable textual inputs rather than calculating or revising them itself.

The implementation uses Python with pinned dependencies, including pandas 3.0.2, yfinance 1.3.0, Streamlit 1.56.0, Ollama 0.6.1 and pytest 9.0.3. The README recommends Python 3.11 and a virtual environment. Ollama is optional at runtime: the quantitative interface remains usable when no local model server is running. This distinction proved important during evaluation because the source and test doubles could be inspected and tested without assuming that a live model was present.

The repository has two modular packages beneath `src`: `engine`, for market-data and indicator operations, and `llm`, for prompt and model-server operations. The user-interface package orchestrates them. Public functions exchange ordinary pandas objects, dictionaries and strings, rather than framework-specific state. This separation supports offline tests and helps identify the provenance of each displayed claim. It also limits, although it does not eliminate, the consequences of a generative error: the model can describe a supplied value incorrectly, but it cannot silently alter the value used by the metric cards.

4.2 Data acquisition and persistence

The ingestion layer exposes `download_stock_data`, accepting either one ticker or a sequence, inclusive start information, an end date interpreted by yfinance as exclusive, an output directory and a persistence flag. Each symbol is downloaded independently with `yf.download`. yfinance can return a two-level column index even for a single symbol; `_download_single_ticker` therefore flattens tuples such as `(“Close”, “AAPL”)` to `Close` before downstream processing. This is a necessary compatibility step because the indicator layer expects a single close-price column.

Downloads that return empty frames are discarded. If every requested symbol is empty, the function returns `None`, allowing the interface to issue a comprehensible “no market data” message rather than exposing a pandas error. For a successful single-symbol request, the frame is passed through unchanged after column normalisation. For a multi-symbol company, frames are concatenated under ticker keys and the available Open, High, Low, Close, Adjusted Close and Volume columns are averaged row by row:

```
combined = pd.concat(frames.values(), axis=1, keys=frames.keys())
summary[column] = combined.xs(column, axis=1, level=1).mean(axis=1)
```

This path supports Alphabet, represented in the interface by GOOG and GOOGL. Averaging gives one reproducible company-level series, but it creates a synthetic composite rather than the price of a tradable security. Even Volume is averaged, not summed. The design is therefore suitable for a demonstrator whose unit of selection is the company, but a production investment tool should either expose the share-class choice or provide a more carefully justified aggregation.

By default, successful results are written to `data/{symbols}_historical.csv`. These files provide inspectable local artefacts and support later analysis without losing the data used for a run. They are not, however, a complete audit log: no code associates a saved frame with a particular prompt or model output, and the application does not read the CSV as a cache. Repeating the same request therefore performs another Yahoo Finance download and overwrites the correspondingly named file. Streamlit caching and run-specific metadata remain appropriate future improvements.

The privacy implications of ingestion require precise wording. No API key is embedded in the data-loader, but `yfinance` necessarily contacts Yahoo Finance. `PrivateFin` consequently implements local language-model inference, not a wholly offline or zero-egress application. Ticker and date choices can be reflected in the market-data request. Risk-profile context is only sent to the local model path in the inspected implementation.

4.3 Indicator engine

The indicator engine converts its close-price input to a floating-point pandas Series and rejects a multi-column DataFrame. `compute_rsi` derives daily changes, clips them into gains and losses, and applies exponentially weighted means with `alpha=1/period`, `adjust=False` and `min_periods=period`. It then computes the familiar relative-strength transformation:

```
average_gain = gains.ewm(alpha=1 / period, adjust=False,
                        min_periods=period).mean()
average_loss = losses.ewm(alpha=1 / period, adjust=False,
                        min_periods=period).mean()
rsi = 100 - (100 / (1 + average_gain / average_loss))
```

Zero-loss observations are replaced with RSI 100 and zero-gain observations with RSI 0; remaining missing warm-up values are filled with 50. This prevents undefined values from reaching the interface or prompt. It also defines a specific convention that must be disclosed when comparing libraries. In a completely flat series both average gain and average loss are zero, and the order of the two replacements yields 0 rather than a neutral 50. The case is unusual for a non-trivial equity history, but it is a genuine residual edge case.

`compute_macd` calculates exponentially weighted moving averages with spans 12 and 26, subtracts the slow series from the fast series, and smooths the result with span 9. The histogram is defined directly as MACD minus its signal line. pandas uses `adjust=False`, and no values are dropped for an explicit warm-up period. These initialisation decisions are internally consistent and deterministic for a supplied frame, but they are not guaranteed to duplicate every external library's seeding convention on a short history.

`enrich_with_indicators` copies an OHLCV frame and appends `RSI_14`, `MACD`, `MACD_Signal` and `MACD_Histogram`. It raises a `ValueError` if `Close` is absent. `summarize_latest_indicators` then creates the contract used by both the interface and prompt builder: latest close, RSI, MACD, signal, histogram, one-session percentage change, trend and momentum. Trend is “bullish” when MACD exceeds signal, “bearish” when it is lower and “neutral” at equality. Momentum is

“overbought” at RSI greater than or equal to 70, “oversold” at less than or equal to 30 and “balanced” otherwise. These labels are heuristic descriptions of technical indicators, not forecasts of return.

The helper `_scalar_from_row` handles a lookup that unexpectedly produces a Series, selecting its first element before conversion to `float`. This defensive behaviour prevents grouped or duplicated columns from leaking non-scalar objects into formatting code. It is pragmatic, although silently choosing the first duplicate could conceal malformed input; explicit validation would be safer in a higher-assurance system.

4.4 Structured prompt construction

The prompt layer operationalises the Observe–Interpret–Infer–Recommend (OIIR) design. A module-level system prompt asks for exactly those four headings, restricts claims to supplied data, requests practical and cautious language, and asks the model to explain jargon. It also instructs the model not to reveal hidden reasoning or internal chain-of-thought. OIIR should therefore be understood as a user-facing explanatory scaffold, not as access to the model’s latent reasoning process.

`build_advisory_prompt` inserts the selected company, source symbols, date range, number of observations, risk profile, latest close, one-session change, RSI, MACD, signal and histogram. It adds deterministic plain-English descriptions generated from the engine’s labels. For example, a bullish classification becomes “recent momentum is leaning upward”, while balanced RSI becomes “the price action looks fairly balanced”. The approach provides both numbers and an interpretation anchor, reducing the need for the model to infer every threshold unaided.

The builder returns three strings: `system`, `user` and a concatenated `prompt` for inspection or debugging. The Streamlit implementation uses the first two but does not display the combined prompt. The README’s statement that the application will “build and display” the structured prompt is therefore not borne out by the current interface source. Surfacing the exact prompt would improve auditability, provided that it is presented as input provenance rather than as model reasoning.

Risk profile is currently a label—`low`, `medium` or `high`—inserted into a common template. There are no explicit profile-specific rules controlling recommendation strength, time horizon or loss tolerance. Consequently, differentiated model behaviour is requested implicitly but not enforced. Furthermore, no generation options fix temperature or seed. Identical prompts may therefore lead to different prose depending on the local model and server configuration.

4.5 Local model client and failure containment

`OllamaClient` is a dataclass whose defaults are model `llama3` and host `http://localhost:11434`. `is_available` creates an SDK client and calls `list`; any exception produces `False`. `generate` constructs explicit system and user chat messages and calls `client.chat`. Response extraction supports both dictionary-shaped and object-shaped SDK responses, which reduces sensitivity to client-version representation changes. Connection or generation failures are re-raised as a `RuntimeError` with instructions to start Ollama and pull the model.

The loopback default is the implementation basis for the local-inference claim. No cloud LLM SDK appears in the inspected source, and the risk profile and generated advisory prompt are directed to localhost. Nevertheless, source inspection cannot establish the absence of all process-level

telemetry, DNS activity or other network traffic during a real session. Nor does it make Yahoo Finance local. Chapter 5 therefore treats privacy as partially demonstrated pending packet capture.

The interface checks availability before generation and shows an OIIR-shaped fallback when Ollama is unavailable. The fallback preserves navigation, chart and metrics, but it is a generic template rather than a generated analysis of the displayed figures. A warning makes this distinction visible. There is a small residual race: a service may pass `is_available` and fail during `generate`; the interface does not catch the later `RuntimeError`, so generation-time failure could still interrupt the Streamlit run.

4.6 Streamlit interface and end-to-end flow

The application configures a wide Streamlit page headed “PrivateFin”, followed by a persistent educational disclaimer. The sidebar contains 108 company options, start and end dates, and Low, Medium and High risk profiles. Most companies map to one ticker; Alphabet maps to two. Dates default to the preceding 365 calendar days and the present day, while Medium is the default risk setting. The application performs no work until “Generate analysis” is selected.

The interface rejects a start date that is equal to or later than the end date. It then downloads prices, enriches them, summarises the latest row and renders four metric cards: close, RSI, MACD and percentage change from the previous observation. A native Streamlit line chart displays Close history. The prompt is assembled with ISO dates and the frame length, after which an available Ollama service produces the OIIR response; otherwise the fallback is shown.

This thin orchestration keeps calculations outside the UI, but several practical limitations remain. The interface has no minimum-window check, so a user can request a history too short for stable interpretation even though warm-up values are syntactically available. It does not expose source data, timestamp the result, display the prompt, store the generated response or let the user choose a model. It analyses one selected company at a time and has no portfolio, holdings, transaction, fundamental-data or profitability component. These boundaries are consistent with a focused prototype, but they constrain claims about “financial advice”: the output is indicator-based educational commentary rather than personalised, regulated advice.

4.7 Verification implemented in the repository

Four test modules contain 18 pytest tests. Data-loader tests use monkeypatching to verify `MultilIndex` flattening, arithmetic averaging for multi-symbol data, `None` on wholly empty downloads and CSV persistence. Indicator tests establish RSI bounds, the identity `histogram = MACD - signal`, expected enrichment columns, missing-Close rejection, trend and momentum classification, percentage change and empty-frame rejection. Prompt tests cover all description branches and confirm that identifying context, risk, values and OIIR wording are present. Ollama tests replace the SDK client to test availability success and failure, system/user message construction, exception translation and both response representations.

These are focused unit tests rather than exhaustive validation. They do not establish known reference values for RSI or MACD, exercise a live Yahoo Finance response, run a real Llama model, test the Streamlit widget flow or measure whether model prose is faithful. There is also no configured continuous-integration workflow in the inspected material. Independent indicator comparison and the unavailable live-model studies are therefore reported separately in Chapter 5.

The implemented core nevertheless forms a coherent demonstrator: public market prices are normalised, deterministic indicator values are calculated, a bounded explanatory prompt is constructed, and a local endpoint is used when present. Its strongest engineering property is separation of concerns. Its principal technical debt lies in incomplete end-to-end verification, convention-sensitive indicator warm-up, absence of run-level audit persistence, implicit rather than enforceable risk adaptation, and documentation that is not fully synchronised with the current interface.

One further implementation constraint concerns reproducibility. The CSV output preserves prices, but it does not record package versions, retrieval time, user settings, prompt text, model digest or generated answer. Reconstructing a historical session would therefore require assumptions beyond the saved file. A compact manifest linking these artefacts would turn the existing persistence mechanism into a stronger audit trail without changing the analytical method.

Chapter 5: Evaluation

5.1 Aims, evidence and decision rules

Evaluation was organised around five aims rather than a single aggregate measure. First, **implementation reliability** asked whether module behaviour remained consistent under controlled tests. Second, **indicator agreement** asked whether the latest PrivateFin RSI and MACD-family values matched an independent TA-Lib reference to four decimal places. Third, **privacy** asked whether personal query context remained on-device during language-model inference, while acknowledging the separate Yahoo Finance connection. Fourth, **explanation quality** asked whether generated OIIR text was factually faithful, coherent, complete and transparently linked to evidence. Fifth, **usability** asked whether intended non-specialist users could complete tasks and understand the explanation.

No method covers all aims: tests assess contracts, TA-Lib assesses numerical agreement, traffic capture assesses observed egress, rubric scoring assesses text, and SUS and interviews assess reception. Missing studies are recorded as unavailable. Four-decimal indicator agreement is treated as agreement with TA-Lib 0.7.1, not an infallible bug detector, because finite-series seed conventions can differ.

5.2 Aim 1: implementation reliability

Method

The test suite was executed locally with pytest on 21 July 2026. It contains four data-loader, six indicator-engine, three prompt-builder and five Ollama-client tests. Network and model dependencies are replaced by deterministic frames and monkeypatched clients, making the run reproducible without Yahoo Finance or Ollama. The decision rule was that all committed tests should pass.

Results

All **18 of 18 pytest tests passed on 21 July 2026**. The result supports the specified behaviours described in Section 4.7: frame normalisation and blending, CSV writing, indicator invariants and errors, summary classification, required prompt content, and Ollama-client message and exception handling.

Interpretation

The outcome demonstrates internal consistency, not complete correctness. The RSI test verifies bounds rather than a canonical sequence, the MACD test verifies an internal relationship, and no test clicks through Streamlit. Independent numerical validity therefore depends on the TA-Lib study.

Threats

Tests may reproduce implementation assumptions; mocks simplify external behaviour, coverage was not supplied, and no CI record demonstrates another platform.

5.3 Aim 2: independent indicator agreement

Method

A completed cross-validation study compared PrivateFin with **TA-Lib 0.7.1**. Ten ticker groups were selected: AAPL, MSFT, TSLA, JPM, JNJ, XOM, GOOG/GOOGL, AMZN, NVDA and KO. They cover several sectors and volatility profiles, with GOOG/GOOGL exercising PrivateFin's blended-share-class path. Each group was fetched for 365-, 182- and 90-calendar-day windows ending at the fixed exclusive date 20 July 2026. The resulting samples contained approximately 250, 124 and 61 trading observations respectively.

For every ticker-window combination, four latest values were compared: RSI(14), MACD(12,26), the nine-period MACD signal, and MACD histogram. Thus 10 groups $\times$ 3 windows $\times$ 4 metrics produced **120 metric comparisons**. The CSV also records latest absolute difference, a Boolean four-decimal agreement result, mean absolute error over the overlapping valid observations and maximum absolute error. The test compared the same downloaded prices, including the synthetic blended Alphabet close series. A comparison was marked as agreement when latest values rounded consistently to four decimal places.

Results

The full planned matrix was completed with **zero fetch failures**. Across all 30 ticker-window observations per metric, four-decimal latest-value agreement was:

- RSI(14): **11/30**;
- MACD: **11/30**;
- MACD signal: **11/30**;
- MACD histogram: **12/30**.

This is 45 agreements among 120 latest-value comparisons overall, so the original universal-agreement criterion was not met. The aggregate result alone obscures a pronounced window-length pattern. **All 40 latest comparisons on the 365-day windows agreed to four decimal places**: ten ticker groups multiplied by four metrics. The additional five agreements came from shorter windows. The CSV shows, for example, that AAPL's 365-day RSI differs by approximately (2.20×10^{-7}) , whereas its 90-day RSI differs by approximately 0.487. Likewise, its 365-day MACD difference is approximately (8.28×10^{-8}) , compared with approximately 0.242 over 90 days.

The convergence is not isolated to one security. Every 365-day row is marked true for every metric, including the blended GOOG/GOOGL series. In contrast, most 90-day latest comparisons are false at four decimals. The same CSV records larger errors near the beginning of overlaps than at the end of the longest series: for example, the latest 365-day values can be nearly indistinguishable while maximum overlap errors remain materially larger. This temporal shape is consistent with initial values decaying as additional observations are processed.

Interpretation

The evidence supports two conclusions. First, PrivateFin is not equivalent to TA-Lib 0.7.1 at four decimals for arbitrary short input histories; the criterion must be judged not achieved across the planned matrix. Second, the complete 365-day agreement shows that the formulas converge to the same latest values once sufficient prior history is supplied. The most plausible explanation is

differing initialisation or seed conventions. PrivateFin applies `pandas ewm` recursively from the beginning of the supplied frame and fills RSI warm-up values, whereas TA-Lib applies its own lookback and seeding procedures. Shortening the input changes the state from which recursive averages begin, so even the same algebra can produce a different endpoint.

These divergences should **not** be declared definitely to be software bugs. They reveal a reproducibility condition: PrivateFin's latest indicators depend not only on ticker and endpoint but also on the amount of preceding data supplied. Whether TA-Lib's convention should replace the current one is a design decision requiring an explicit reference standard. A practical correction is to fetch a fixed pre-roll period—longer than the user-visible range—compute indicators on that history, and then crop the displayed window. This would reduce initialisation sensitivity while preserving the user's requested chart range. Alternatively, the report and UI could declare the pandas convention and minimum recommended history.

The study materially strengthens the evaluation because it goes beyond internal invariants. It also narrows the correctness claim: long-window latest values agree to four decimals for this sample and date, while short-window agreement with TA-Lib is partial. No claim is made about predictive profitability, because RSI and MACD agreement measures calculation, not whether an advisory action earns a return.

Threats

The purposive US-equity sample and single endpoint do not cover illiquid instruments, other assets or multiple regimes. TA-Lib is a reference implementation, not absolute ground truth, and Yahoo Finance revisions may alter results. Headline counts concern latest values; overlap errors show different early-series behaviour.

5.4 Aim 3: local-inference privacy

Method

Ten frozen prompts from the OIIR study were submitted directly to `llama3:latest` at `localhost:11434`, excluding the preceding Yahoo Finance retrieval so that the inference phase could be isolated. A monitor executed `lsof -nP -i` approximately every 150 milliseconds and retained sockets owned by the evaluation Python process and Ollama server. Each request was capped at 64 generated tokens. The criterion was zero external sockets owned by either relevant process during the ten requests.

This was an endpoint audit rather than payload-level packet capture. It can identify destination addresses and distinguish loopback from external connections without elevated capture permission, but it cannot search encrypted packet content and could miss a connection shorter than the sampling interval. Source inspection provided a complementary architectural check: `OllamaClient` defaults to `http://localhost:11434`, and no cloud LLM client appears in the application source.

Results

The audit ran from 10:32:12 to 10:33:10 UTC on 21 July 2026 and collected **273 endpoint snapshots**. All **10/10 queries returned a response**, with individual generation times from approximately 5.4 to 6.8 seconds. The observed sockets comprised the Python client connection to `127.0.0.1:11434`, Ollama's listener on that address, and Ollama's loopback connections to its local

runner. **No external socket was observed for either relevant process.** The pre-declared endpoint criterion was therefore met for this sample.

Interpretation

The endpoint evidence strengthens the claim that live prompt inference remained local in the tested configuration. It is consistent with the source boundary and observed only loopback communication across ten real generations. It does not justify “zero system egress”: Yahoo Finance is contacted during data acquisition, model installation requires a download, and this audit did not inspect packet payloads. The defensible conclusion is that **no external inference-process endpoint was observed**, not that all possible runtime configurations are cryptographically isolated.

Threats

The monitor sampled rather than captured continuously, followed only the identified Python and Ollama processes, and did not inspect UDP, packet payloads or unrelated helper processes. A privileged Wireshark or `tcpdump` study remains a useful replication. Model, operating-system or SDK changes could alter behaviour.

5.5 Aim 4: OIIR explanation quality and faithfulness

Method

Twenty live outputs were generated using `llama3:latest` (8.0B, Q4_0; recorded model digest) through local Ollama. The ten ticker groups used by the TA-Lib study were each evaluated with Low and High risk labels over the fixed 365-day period. Four dimensions were scored from zero to two: factual accuracy, logical coherence, OIIR completeness and data-linked transparency. The maximum was eight and the pre-declared target mean was 6/8.

Automated checks tested presence of all four headings, exact citation of the two-decimal RSI value, exact citation of the four-decimal MACD value when used, and absence of an unnegated RSI threshold contradiction. The author then scored every frozen response using the documented rubric. No independent second scorer was available, so inter-rater reliability is not claimed.

Results

All **20/20 outputs contained the four OIIR headings**, cited the expected RSI value and avoided an obvious unnegated momentum-threshold contradiction. The exact MACD value appeared in **13/20** responses; omission was not automatically treated as an error where the line/signal relation was described. These rule checks show strong format compliance and numeric copying, but they do not establish semantic correctness.

The manual totals had a **mean of 5.95/8**, median 6, range 3–8, with 14/20 samples scoring at least 6. The pre-declared mean threshold was therefore narrowly **not met**. Dimension means were 1.35/2 for factual accuracy, 1.10/2 for logical coherence, 2.00/2 for completeness and 1.50/2 for transparency.

Errors were substantive rather than cosmetic. MSFT Low and GOOG/GOOGL High each scored 3/8 after reversing the order or meaning of negative MACD and signal values. AMZN High recommended allocating 60–70% of a portfolio despite mixed evidence and no portfolio information, scoring 4/8. Other outputs invented target prices, stop levels, holding assumptions or

portfolio percentages absent from the prompt. Conversely, TSLA High, JPM High, KO Low and KO High scored 8/8 under the author rubric.

Interpretation

The study confirms that OIIR reliably controls response structure but does not reliably control inference quality. Perfect completeness coexists with repeated sign/order errors and unsupported specificity. Numeric copying is easier for the model than reasoning correctly about the relation between two negative values. The risk-profile label also failed to constrain advice: some High responses remained cautious while one proposed extreme concentration.

These results motivate deterministic post-generation checks of MACD-versus-signal language, rejection of invented price targets or portfolio percentages, explicit RSI boundary text, and rule-based risk constraints. A revised prompt should be evaluated on a separate frozen sample rather than replacing these baseline results.

Threats

The study used one model build, one period and a single author scorer. Generation settings were not fully fixed by the application, and no inter-rater statistic is available. The automatic momentum check handles local negation but cannot determine semantic nuance. Scores therefore support formative critique rather than general claims about all Llama 3 outputs.

5.6 Aim 5: usability with target users

Method

The prepared protocol recruits at least five adults resembling novice or general retail investors without collecting holdings, income or account data. After consent, participants complete standardised tasks: generate an Apple analysis with the default one-year range and Medium risk; locate and explain RSI and OIIR momentum; then select another company and risk profile. The observer records task completion, significant errors, help requests and duration. Participants rate clarity and appropriate trust on five-point scales, complete the ten-item System Usability Scale (SUS), and answer six semi-structured questions. Pre-declared targets are mean SUS at least 70 and mean clarity at least 4/5.

Results

User-study results remain pending real participants. No participant rows, SUS scores, clarity scores, timing data, quotations or themes are available, and none are inferred from implementation quality.

Interpretation

The existence of a browser interface, plain-language prompt and fallback supports a claim that the system was designed for non-technical access. It does not show that users actually understand RSI, distinguish local inference from external market-data retrieval, or calibrate trust appropriately. Usability and transparency objectives are therefore not demonstrated empirically. The study should remain small-sample formative evaluation rather than being presented as population-level evidence.

Threats

Convenience recruitment, researcher presence, financial knowledge and variable model outputs can affect results. SUS measures perceived usability, not correctness. Ethics requirements and consent for quotations must be resolved before recruitment.

5.7 Integrated interpretation and limitations

Evaluation provides strongest evidence for the deterministic software core, qualified evidence for reference agreement, and direct but bounded evidence for the network and explanation claims. Eighteen passing tests show that specified module contracts work under controlled conditions. The 120-comparison TA-Lib study is more discriminating: universal 365-day latest agreement supports long-history convergence, while 182- and 90-day differences reject unconditional equivalence. The result points to initialisation sensitivity, not a demonstrated algebraic defect.

Privacy is supported by the loopback architecture and by 273 endpoint snapshots across ten successful inferences with no external process socket observed; payload-level capture remains absent. Explainability was tested at the planned 20-output scale, but the 5.95/8 author-scored mean missed the 6/8 target and exposed MACD reasoning errors despite perfect OIIR completeness. Usability has a protocol but no participants. These distinctions prevent implementation completion from being mistaken for objective achievement.

The evidence also illustrates why a binary project-wide pass/fail label would be misleading. Reliability evidence concerns code paths under controlled inputs; numerical evidence concerns agreement with one external implementation; privacy evidence currently concerns architecture; and explanation evidence concerns only three historical outputs. Each supports a different inference. The application can therefore be described as operational without claiming that its advice is validated. Similarly, four-decimal convergence on long histories strengthens confidence in endpoint calculations but does not validate the economic meaning of RSI or MACD, the model's interpretation of them, or the suitability of any recommendation for an individual.

The next evaluation sequence should preserve these boundaries. First, indicator pre-roll should be fixed and the 120 comparisons repeated, retaining the present CSV as a baseline. Second, payload-level capture should replicate the endpoint audit. Third, deterministic MACD and unsupported-target checks should be added before a separately labelled post-revision OIIR study with an independent scorer. Finally, participant sessions should use the documented model configuration. This ordering reduces the risk that iterative improvements contaminate comparisons.

Cross-cutting limitations include revisable Yahoo Finance data, unfixed generation parameters, one application configuration and author involvement in rubric scoring. Profitability was not evaluated because no trading strategy is executed.

Chapter 6: Conclusion

6.1 Project outcome

PrivateFin demonstrates a coherent local-first architecture for turning public equity prices into inspectable technical indicators and a structured natural-language explanation. The implemented Streamlit workflow covers company and date selection, risk-profile context, Yahoo Finance retrieval, RSI and MACD calculation, current metrics, a price chart, OIIR prompt construction and optional Llama 3 generation through Ollama on localhost. A generic fallback retains access to the quantitative interface when Ollama is unavailable. The central contribution is not a new forecasting algorithm, but an explicit separation between deterministic quantitative evidence and a generative presentation layer.

The evaluation supports a qualified account of success. The working application and modular core are **achieved**, and all 18 tests passed on 21 July 2026. Independent numerical validation is **partially achieved**: TA-Lib 0.7.1 comparisons completed for ten ticker groups, three windows and 120 metrics with no fetch failures. Every 365-day latest value agreed to four decimals, but aggregate agreement was only 11/30 for RSI, MACD and signal and 12/30 for histogram. Short-window differences are consistent with initialisation conventions and are not sufficient evidence of bugs.

The privacy objective is **substantially but not completely demonstrated**. Source code directs model prompts to loopback, and a 273-snapshot endpoint audit across ten successful inferences observed no external socket for the Python or Ollama processes. Yahoo Finance remains external and packet payloads were not captured, so the intended claim remains local LLM inference rather than zero system egress. Explainability is **partially demonstrated**: all 20 formal outputs followed OIIR structure, but the author-scored mean of 5.95/8 narrowly missed the target and several responses reversed MACD relations or invented unsupported targets. Usability with target users is **not demonstrated**, because real participant results remain pending.

6.2 Contribution, limitations and future work

PrivateFin's value lies in integrating modest, auditable components around a defensible boundary of authority. pandas supplies numeric outputs; the model is asked to explain them; and the interface labels the result educational. This architecture is more inspectable than asking an unconstrained model to invent both analysis and evidence. It is nevertheless not a professional adviser: it analyses one company, uses two lagging technical indicators, does not consider fundamentals, portfolios or personal circumstances, and does not validate recommendations through returns.

The highest-priority work is evidential and corrective rather than cosmetic. Payload-level capture should replicate the successful endpoint audit. Deterministic guards should check MACD ordering and reject unsupported targets before a second 20-output study with fixed settings and an independent scorer. Real participants should complete the prepared usability protocol. Indicator calculation should use a declared pre-roll or minimum history so visible date choices do not create avoidable seed sensitivity. Prompt display, generation metadata and a run-level audit record would strengthen traceability; caching and UI tests would improve reliability.

The final objective scorecard is explicit:

- **Achieved:** a working single-company Streamlit application; modular data, indicator, prompt and local-client layers; graceful no-Ollama fallback; and controlled unit-test reliability.
- **Partially achieved:** deterministic correctness, because long-window TA-Lib agreement was complete but short-window agreement was convention-sensitive; and local-inference privacy, because the endpoint audit passed but packet contents were not inspected.
- **Partially demonstrated only:** structured explainability, because completeness was 20/20 but the formal author-scored mean was 5.95/8 and important factual errors remained.
- **Not demonstrated:** usability and clarity for real target users, since no participant study has been completed.
- **Outside the demonstrated scope:** investment profitability, portfolio suitability and regulatory compliance as a professional advisory service.

6.3 Final judgement

The project achieved a functioning, testable local-first prototype, demonstrated long-window agreement with an independent numerical library, and observed only loopback process endpoints during ten live inferences. It partially achieved deterministic correctness and privacy objectives under bounded claims. Explanation quality was tested at scale and failed the pre-declared mean threshold narrowly, revealing that reliable formatting does not equal faithful reasoning; user usability has not yet been demonstrated. PrivateFin therefore establishes an implementable foundation while identifying concrete verification and prompt-safety work still required.

Accordingly, the prototype should be judged as a completed implementation with an incomplete empirical case, rather than as either a failed project or a validated advisory product. That distinction is the most important conclusion of the work.

References

- Appel, G. (1979) *The Moving Average Convergence-Divergence Method*. Signalert Corporation.
- Araci, D. (2019) 'FinBERT: Financial Sentiment Analysis with Pre-trained Language Models', *arXiv preprint*, arXiv:1908.10063.
- Aroussi, R. (2019) *yfinance*. Available at: <https://pypi.org/project/yfinance/> (Accessed: 21 July 2026).
- Bender, E.M., Gebru, T., McMillan-Major, A. and Shmitchell, S. (2021) 'On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?', **Proceedings of FAccT 2021**, pp. 610–623.
- Brooke, J. (1996) 'SUS: A "quick and dirty" usability scale', in Jordan, P.W. et al. (eds.) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189–194.
- Devlin, J. et al. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', *NAACL-HLT 2019*, pp. 4171–4186.
- European Union (2016) **Regulation (EU) 2016/679 (General Data Protection Regulation)**. Official Journal of the European Union, L 119. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Accessed: 21 July 2026).
- European Union (2024) *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*. Official Journal of the European Union, L series. Available at: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj> (Accessed: 21 July 2026).
- Grattafiori, A. et al. (2024) 'The Llama 3 Herd of Models', *arXiv preprint*, arXiv:2407.21783.
- Hu, E.J. et al. (2022) 'LoRA: Low-Rank Adaptation of Large Language Models', *Proceedings of ICLR 2022*, arXiv:2106.09685.
- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.
- Malo, P. et al. (2014) 'Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts', *Journal of the Association for Information Science and Technology*, 65(4), pp. 782–796.
- Manning, C.D., Raghavan, P. and Schütze, H. (2008) **Introduction to Information Retrieval**. Cambridge: Cambridge University Press.
- McKinney, W. (2010) 'Data Structures for Statistical Computing in Python',

Proceedings of SciPy 2010, pp. 56–61.

Ollama (2023) *Ollama*. Available at: <https://ollama.com> (Accessed: 21 July 2026).

Rudin, C. (2019) ‘Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead’, *Nature Machine Intelligence*, 1(5), pp. 206–215.

Streamlit (2019) *Streamlit*. Available at: <https://streamlit.io> (Accessed: 21 July 2026).

Wei, J. et al. (2022) ‘Chain-of-Thought Prompting Elicits Reasoning in Large Language Models’, *arXiv preprint*, arXiv:2201.11903.

Wilder, J.W. (1978) *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research.

Wu, S. et al. (2023) ‘BloombergGPT: A Large Language Model for Finance’, **arXiv preprint**, arXiv:2303.17564.

Yang, H. et al. (2023) ‘FinGPT: Open-Source Financial Large Language Models’, **arXiv preprint**, arXiv:2306.06031.